

Entregable segundo corte



Universidad
del Cauca

Vigilada Mineducación

Ingeniería de software II

Presentado por:

Brayan Steven Charry Vela
Jhoiner Alberto Puentes Figueroa
Sebastian Ruiz Segura

Profesor:

Wilson Libardo Pantoja Yepez

Universidad del Cauca

Ingeniería electrónica y telecomunicaciones

Ingeniería de sistemas

Popayán, mayo

Tabla de Contenido

- Introducción
- Historias de usuario segundo sprint
- Planificación de Tareas en Jira
- Escenario de calidad escalabilidad
- Modelo C4
- Listado de patrones de diseño
- URL del video de YouTube:
- URL del repositorio GIT

1. Introducción

El presente documento describe la arquitectura de software desarrollada para la segunda iteración del sistema PiedraAzul, correspondiente al proyecto de Ingeniería de Software II. En esta etapa se realizó una evolución del sistema construido en la primera iteración, incorporando una arquitectura basada en microservicios y comunicación orientada a eventos, con el propósito de mejorar la organización, mantenibilidad, escalabilidad y separación de responsabilidades del sistema.

Durante esta segunda iteración se implementan funcionalidades relacionadas con la gestión de disponibilidad de médicos y terapeutas, el agendamiento autónomo de citas por parte de los pacientes y el agendamiento manual realizado por usuarios autorizados. Estas funcionalidades permitieron ampliar el alcance del sistema y fortalecer el proceso de atención médica mediante una estructura más modular y desacoplada.

El documento presenta las historias de usuario implementadas junto con sus criterios de aceptación, la planificación del Sprint 2 mediante Jira, el escenario de calidad asociado a la escalabilidad, la arquitectura del sistema representada mediante el modelo C4, los microservicios definidos, los patrones de diseño GoF aplicados y los enlaces correspondientes al repositorio Git y al video de sustentación.

Con este documento se busca evidenciar las decisiones arquitectónicas tomadas durante la segunda iteración, justificando cómo la solución propuesta responde a los requerimientos funcionales y no funcionales del sistema PiedraAzul.

2. Historias de usuario segundo sprint

HistoriasUsuario

Identificador (ID) de la historia	Rol	Característica / Funcionalidad (Quiero)	Razón / Resultado (Para)	Número (#) de escenario	Criterio de aceptación (Título)	Contexto	Evento	Resultado / Comportamiento esperado
HE-04-HU1	Yo como PACIENTE	Consultar citas disponibles por médico y fecha	Elegir un horario adecuado	1	Horarios disponibles	En caso de que el paciente selecciona un médico y elige una fecha	Clic en consultar disponibilidad	El sistema muestra los horarios disponibles
				2	Sin disponibilidad	En caso de que no hay disponibilidad	Clic en consultar disponibilidad	El sistema muestra mensaje informativo
				3	Formato tabla	En caso de que existen horarios	Clic en consultar disponibilidad	Los horarios se listan en formato tabla
				4	Médico no seleccionado	En caso de que el paciente no selecciona un médico	Clic en consultar disponibilidad	El sistema muestra mensaje indicando que debe seleccionar un médico
				5	Fecha no seleccionada	En caso de que el paciente no selecciona una fecha	Clic en consultar disponibilidad	El sistema muestra mensaje indicando que debe seleccionar una fecha
HE-04-HU2	Yo como PACIENTE	Agendar una cita desde la web	Obtener atención sin intermediarios	1	Confirmar cita	En caso de que el paciente está registrado y selecciona un horario disponible	Seleccionar horario disponible	El paciente puede confirmar la cita
				2	Horario tomado	En caso de que el horario ya fue tomado	Intentar confirmar horario	El sistema rechaza la solicitud
				3	Cita guardada	En caso de que la cita es creada	Confirmar la cita	La cita se guarda correctamente en el sistema
				4	Paciente no registrado	En caso de que el paciente no está registrado en el sistema	Intentar agendar una cita	El sistema no permite agendar y solicita registro
				5	Notificación de cita	En caso de que la cita es confirmada exitosamente	Confirmar la cita	El sistema envía notificación de confirmación al paciente

HE-05-HU3	Yo como AGENDADOR	Registrar una cita manualmente	Atender pacientes contactados por otros medios	1	Crear cita	En caso de que el agendador ingresa datos	Completar formulario	El sistema permite crear la cita
				2	Datos faltantes	En caso de que faltan datos obligatorios	Intentar guardar con campos	El sistema muestra error
				3	Horario no disponible	En caso de que el horario no está disponible	Intentar agendar en horario	El sistema no permite agendar
				4	Paciente no encontrado	En caso de que el agendador busca un paciente que no está registrado	Buscar paciente en el sistema	El sistema muestra mensaje indicando que el paciente no existe
				5	Cita registrada correctamente	En caso de que todos los datos son válidos y el horario está disponible	Clic en guardar cita	El sistema guarda la cita y la muestra en la agenda del médico
HE-05-HU4	Yo como AGENDADOR	Ver las citas de un médico en una fecha	Controlar la agenda	1	Listar citas	En caso de que se selecciona un médico y una fecha con citas registradas	Clic en consultar agenda	Se listan las citas del médico para esa fecha
				2	Sin citas	En caso de que no hay citas para ese médico y fecha	Clic en consultar agenda	Se muestra mensaje vacío
				3	Médico no seleccionado	En caso de que el agendador no selecciona un médico	Clic en consultar agenda	El sistema muestra mensaje indicando que debe seleccionar un médico
				4	Fecha no seleccionada	En caso de que el agendador no selecciona una fecha	Clic en consultar agenda	El sistema muestra mensaje indicando que debe seleccionar una fecha
				5	Formato tabla agenda	En caso de que existen citas para el médico y fecha seleccionados	Clic en consultar agenda	Las citas se muestran en formato tabla con hora, paciente y estado

HE-01-HU5	Yo como ADMINISTRADOR	Definir horarios y disponibilidad de médicos	Controlar el agendamiento	1	Guardar horarios	En caso de que el admin define los horarios de atención	Clic en guardar configuración	Los horarios se guardan correctamente
				2	Respetar intervalos	En caso de que el admin define intervalos de atención	Clic en guardar intervalos	El sistema respeta los intervalos definidos
				3	Día sin atención	En caso de que un médico no atiende un día determinado	Consultar disponibilidad de ese médico ese día	No aparecen citas disponibles para ese médico
				4	Intervalo no válido	En caso de que el admin ingresa un intervalo de tiempo inválido o en conflicto	Clic en guardar intervalos	El sistema muestra mensaje de error indicando el conflicto
				5	Editar disponibilidad	En caso de que el admin quiere modificar la disponibilidad ya configurada de un médico	Clic en editar disponibilidad	El sistema permite actualizar los datos y los guarda correctamente
				6	Médico sin horario	En caso de que el admin no ha definido horario para un médico	Consultar disponibilidad de ese médico ese día	El sistema no muestra horarios disponibles para ese médico

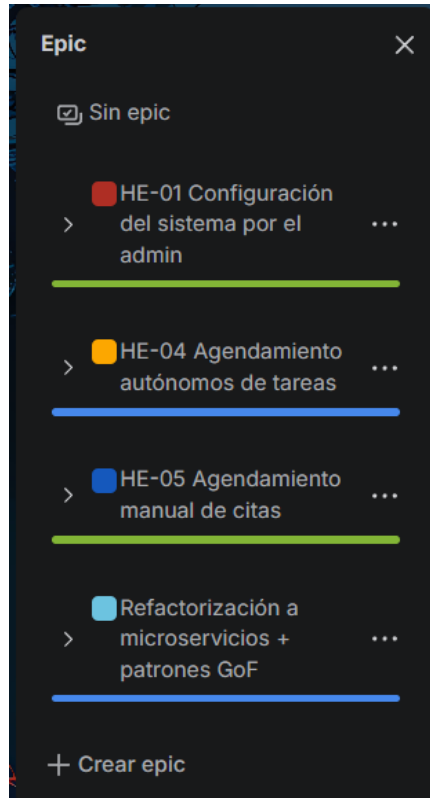
3. Planificación de Tareas del Sprint 2

El Sprint 2 fue gestionado mediante Jira como herramienta de planificación y seguimiento del proyecto, permitiendo organizar las actividades relacionadas con la implementación de la arquitectura basada en microservicios y los nuevos requerimientos funcionales definidos para la segunda iteración. Durante este sprint se trabajó principalmente en las funcionalidades de configuración de disponibilidad de médicos y terapeutas, el agendamiento autónomo de citas y la integración de los diferentes microservicios del sistema.

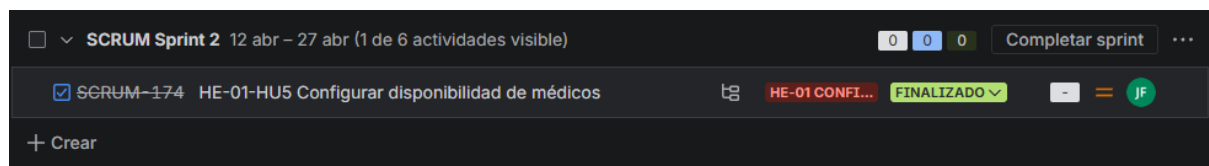
Cada Historia de Usuario fue dividida en tareas y subtareas enfocadas en diferentes etapas del desarrollo, incluyendo diseño de interfaces, definición de criterios de aceptación, implementación de microservicios, creación de endpoints REST, pruebas unitarias y validaciones funcionales. Además, se asignaron responsabilidades específicas a cada integrante del equipo para facilitar el trabajo colaborativo y el control del avance del proyecto.

Por medio del tablero de Jira fue posible hacer seguimiento al estado de cada actividad, identificar tareas pendientes y controlar el cumplimiento de los objetivos planteados para el Sprint 2. Esto permitió mantener una mejor organización del trabajo y asegurar la correcta integración entre los componentes del sistema. Al finalizar la iteración, el tablero reflejó el cumplimiento de las actividades planificadas, evidenciando el desarrollo exitoso de las funcionalidades propuestas y la adaptación de la aplicación a una arquitectura más escalable y desacoplada.

Las épicas que se desarrollaron fueron:



Para la HE-01 Configuración del sistema por el admin tenemos la siguiente historia de usuario:



A continuación se muestran sus subtareas:

Actividad	Prioridad	Persona	Estado
 SCRUM-175 Modelo de horarios	= M.	 Si	FINALIZADO 
 SCRUM-176 Patrón Facade (coordinación del...	= M.	 Si	FINALIZADO 
 SCRUM-177 Patrón Strategy (cálculo de...	= M.	 Si	FINALIZADO 
 SCRUM-178 Persistencia	= M.	 Si	FINALIZADO 
 SCRUM-179 Pruebas	= M.	 Si	FINALIZADO 

Para la HE-04 Agendamiento autónomo de tareas tenemos las siguientes historias de usuario:

☐	SCRUM Sprint 2 12 abr – 27 abr (2 de 6 actividades visible)	0 0 0	Completar sprint	...
☒	SCRUM-152 HE-04-HU1 Consultar disponibilidad de citas	 HE-04 AGEN...	EN CURSO 	- = SS
☒	SCRUM-159 HE-04-HU2 Agendar cita autónoma	 HE-04 AGEN...	EN REVISIÓN 	- = SS
+ Crear				

A continuación se muestran sus subtareas:

Actividad	Prioridad	Persona	Estado
 SCRUM-154 Diseñar endpoint /disponibilidad...	= M.	 Si	FINALIZADO 
 SCRUM-155 Implementar patrón Repository (acces...	= M.	 Si	FINALIZADO 
 SCRUM-156 Crear lógica de cálculo de...	= M.	 Si	FINALIZADO 
 SCRUM-157 Integrar con microservicio de...	= M.	 Si	FINALIZADO 
 SCRUM-158 Pruebas unitarias	= M.	 Si	FINALIZADO 

Actividad		Prioridad	Persona	Estado
 SCRUM-160	Patrón Adapter (integración con...	= M.	 Sii	FINALIZADO 
 SCRUM-161	Patrón Factory Method (creación...	= M.	 Sii	FINALIZADO 
 SCRUM-162	Patrón Observer (evento: cita...	= M.	 Sii	FINALIZADO 
 SCRUM-163	Persistencia en BD	= M.	 Sii	FINALIZADO 
 SCRUM-164	Pruebas de integración	= M.	 Sii	FINALIZADO 

Para la HE-05 Agendamiento manual de citas tenemos las siguientes historias de usuario:

☐	▼	SCRUM Sprint 2	12 abr – 27 abr (2 de 6 actividades visible)	0	0	0	Completar sprint	...
☒		SCRUM-151	HE-05-HU3 Crear cita manual(agendador)		HE-05 AGEN...	FINALIZADO 		= 
☒		SCRUM-169	HE-05-HU4 Listar citas por médico y fecha		HE-05 AGEN...	FINALIZADO 		= 
+ Crear								

A continuación se muestran sus subtareas:

Actividad	...	Prioridad	Persona	Estado
 SCRUM-165	Validaciones de datos del paciente	= M.	 Sii	FINALIZADO 
 SCRUM-166	Patrón Builder (construcción de...	= M.	 Sii	FINALIZADO 
 SCRUM-167	Integración con microservicio de...	= M.	 Sii	FINALIZADO 
 SCRUM-168	Manejo de errores	= M.	 Sii	FINALIZADO 

Actividad	Prioridad	Persona	Estado
 SCRUM-170 Consulta optimizada (índices)	= M.	 Sii	FINALIZADO ▾
 SCRUM-171 "Patrón Builder (construcción de...	= M.	 Sii	FINALIZADO ▾
 SCRUM-172 Formato tabla	= M.	 Sii	FINALIZADO ▾
 SCRUM-173 Pruebas	= M.	 Sii	FINALIZADO ▾

Tareas para la refactorización a microservicios + patrones GoF

☐ **SCRUM Sprint 2** 12 abr – 27 abr (1 de 6 actividades visible)
 0 0 0
Completar sprint
...

☒ SCRUM-181 Arquitectura y patrones GoF
 REFACTORIZ...
EN CURSO ▾
-
=
JF

+ Crear

A continuación se muestran sus subtareas:

Actividad	Prioridad	Persona	Estado
 SCRUM-182 Diseñar arquitectura de microservicios	= M.	 Sii	FINALIZADO ▾
 SCRUM-183 Extraer módulo de citas del monolito	= M.	 Sii	FINALIZADO ▾
 SCRUM-184 Implementar comunicación ent...	= M.	 Sii	FINALIZADO ▾
 SCRUM-185 Implementar patrón Factory Method	= M.	 Sii	FINALIZADO ▾
 SCRUM-186 Implementar patrón Builder	= M.	 Sii	FINALIZADO ▾
 SCRUM-187 Implementar patrón Observer	= M.	 Sii	FINALIZADO ▾
 SCRUM-188 Implementar patrón Strategy	= M.	 Sii	FINALIZADO ▾
 SCRUM-189 Implementar patrón Facade	= M.	 Sii	FINALIZADO ▾
 SCRUM-190 Definir modelo de datos por...	= M.	 Sii	FINALIZADO ▾
 SCRUM-191 Configurar base de datos independien...	= M.	 Sii	FINALIZADO ▾
 SCRUM-192 Pruebas de integración entre...	= M.	 Sii	FINALIZADO ▾

Progreso:

Espero

Sprints

Resumen Backlog Tablero Formularios Informes Calendario Lista +

Q Buscar tablero BV JF SS Filtro

POR HACER

+ Crear

EN CURSO

+ Crear

EN REVISIÓN

FINALIZADO 6 ✓

HE-04-HU2 Agendar cita autónoma

HE-04 AGENDAMIENTO AUTÓNOMOS ...

✓ SERUM-159 ✓ = SS

Arquitectura y patrones GoF

REFACTORIZACIÓN A MICROSERVICI...

✓ SERUM-181 ✓ = JF

HE-04-HU1 Consultar disponibilidad de citas

HE-04 AGENDAMIENTO AUTÓNOMOS ...

✓ SERUM-152 ✓ = SS

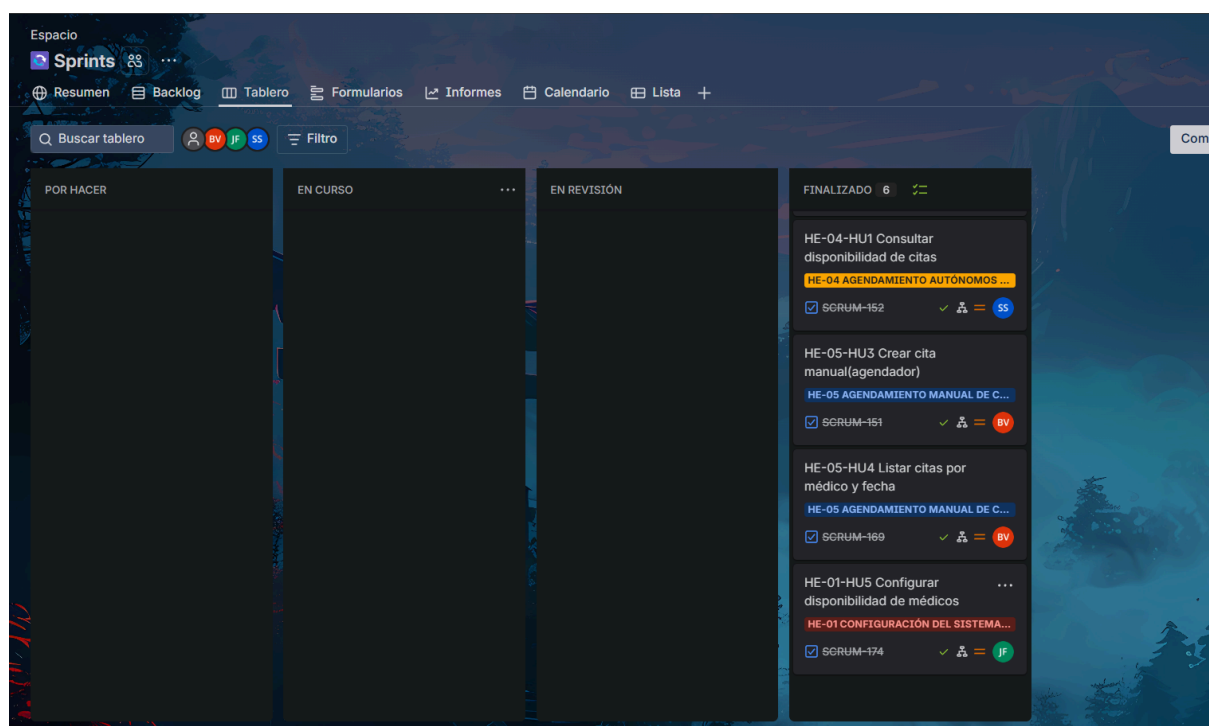
HE-05-HU3 Crear cita manual(agendador)

HE-05 AGENDAMIENTO MANUAL DE C...

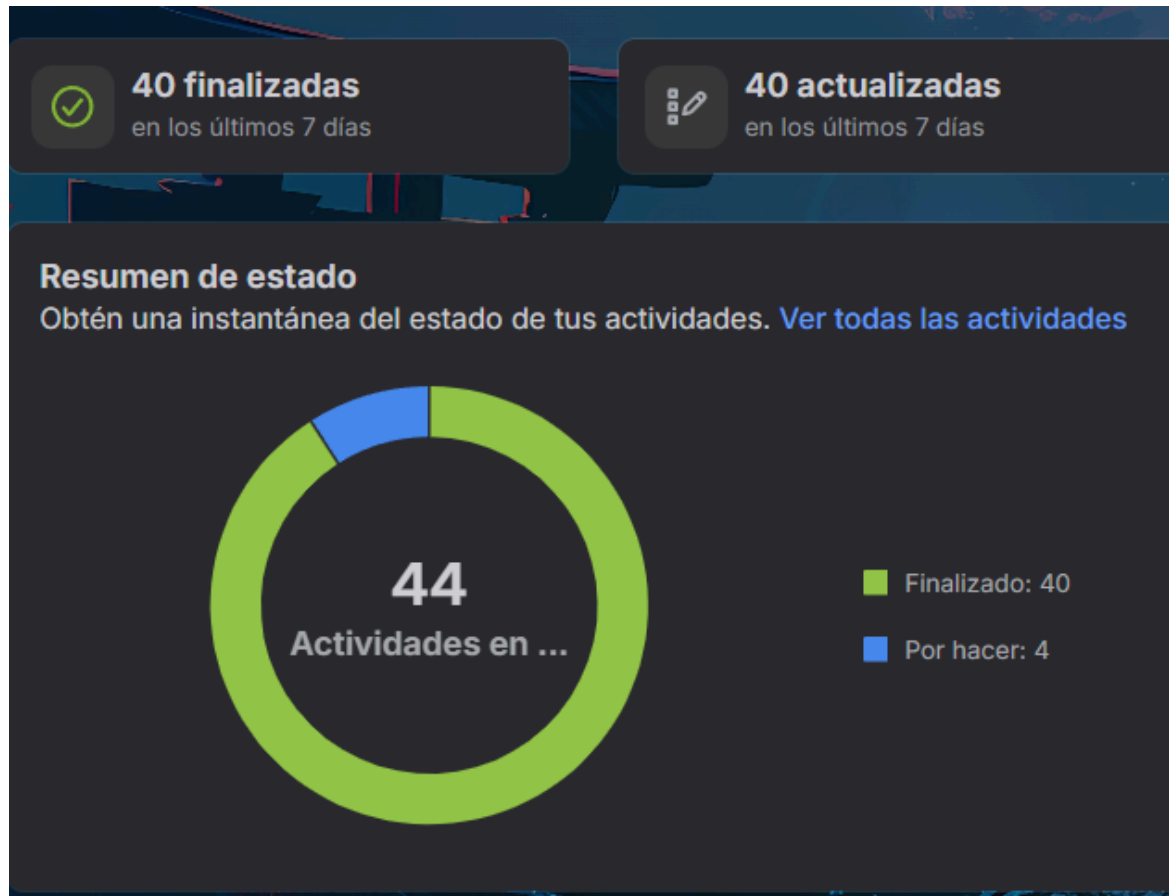
✓ SERUM-151 ✓ = BV

HE-05-HU4 Listar citas por médico y fecha

HE-05 AGENDAMIENTO MANUAL DE C...



Resultado



4. Escenario de Calidad – Escalabilidad

Contexto:

El sistema PiedraAzul se encuentra en funcionamiento durante una jornada de alta demanda, en la cual múltiples pacientes y agendadores consultan la disponibilidad médica y registran citas de manera simultánea desde la aplicación de escritorio.

Estímulo:

Se presenta un aumento considerable en las solicitudes enviadas hacia la API REST del sistema, principalmente en las operaciones de consulta de disponibilidad médica, creación de citas y envío de notificaciones asociadas al agendamiento.

Respuesta:

El sistema procesa las solicitudes de forma estable, manteniendo la comunicación entre el frontend de escritorio, el microservicio de agenda, el microservicio de notificaciones y la base de datos. Además, la separación por microservicios permite que el componente encargado del agendamiento pueda escalar o desplegarse de manera independiente sin afectar directamente al servicio de notificaciones.

Medición de calidad:

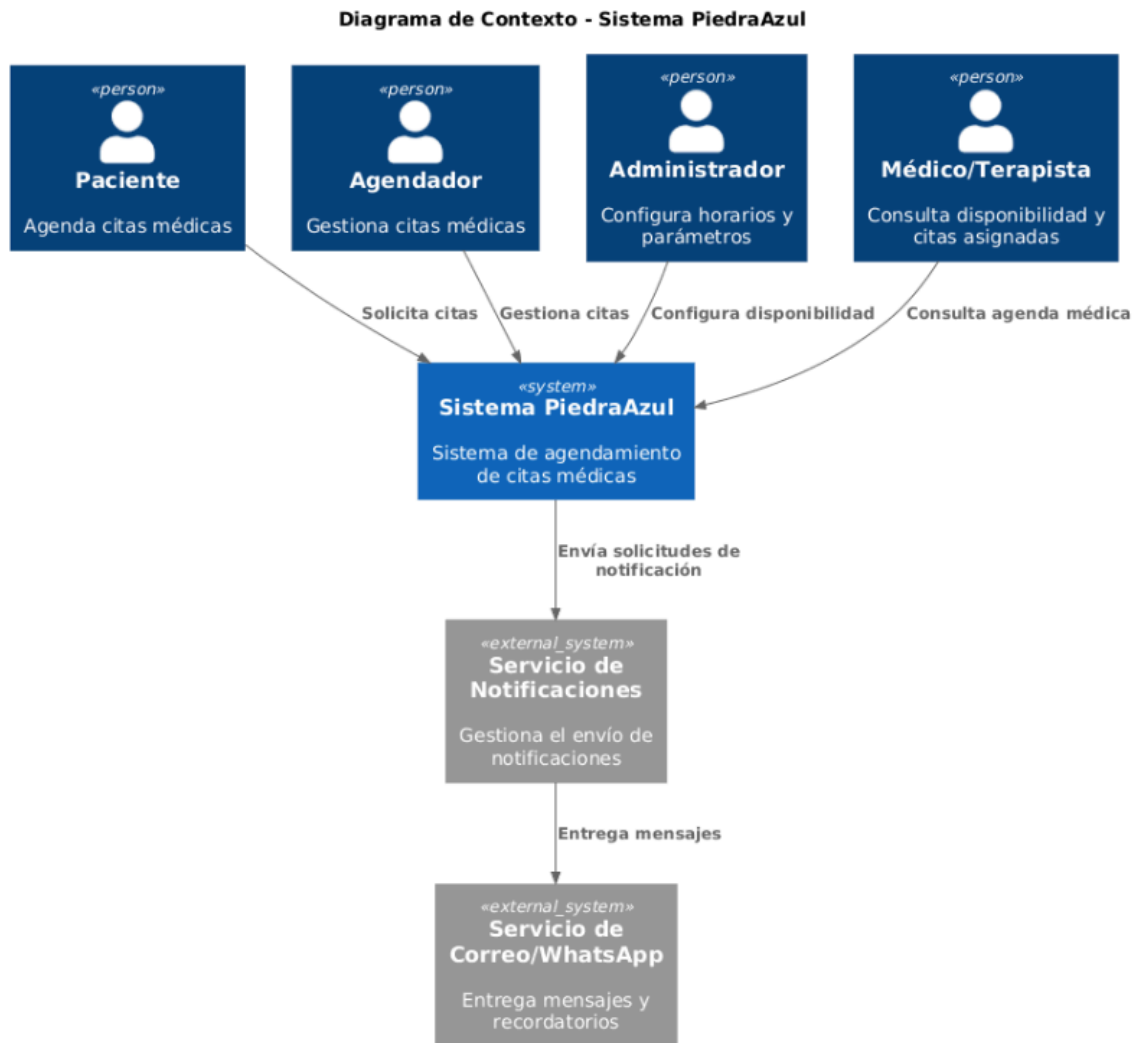
El sistema debe responder las solicitudes de consulta de disponibilidad y registro de citas en un tiempo menor o igual a 3 segundos bajo una carga concurrente de al menos 50 usuarios. Además, el microservicio de agenda debe poder desplegarse o ajustarse de forma independiente sin detener el funcionamiento general del sistema.

Resultado esperado:

La arquitectura basada en microservicios permite soportar un mayor número de solicitudes concurrentes, mejorar la distribución de responsabilidades y facilitar la escalabilidad horizontal del backend. Como resultado, el sistema mantiene una operación estable durante escenarios de alta demanda y reduce el impacto de cambios o ajustes sobre otros componentes del sistema.

5. Modelo C4

Diagrama de Contexto – Modelo C4 (Nivel 1)



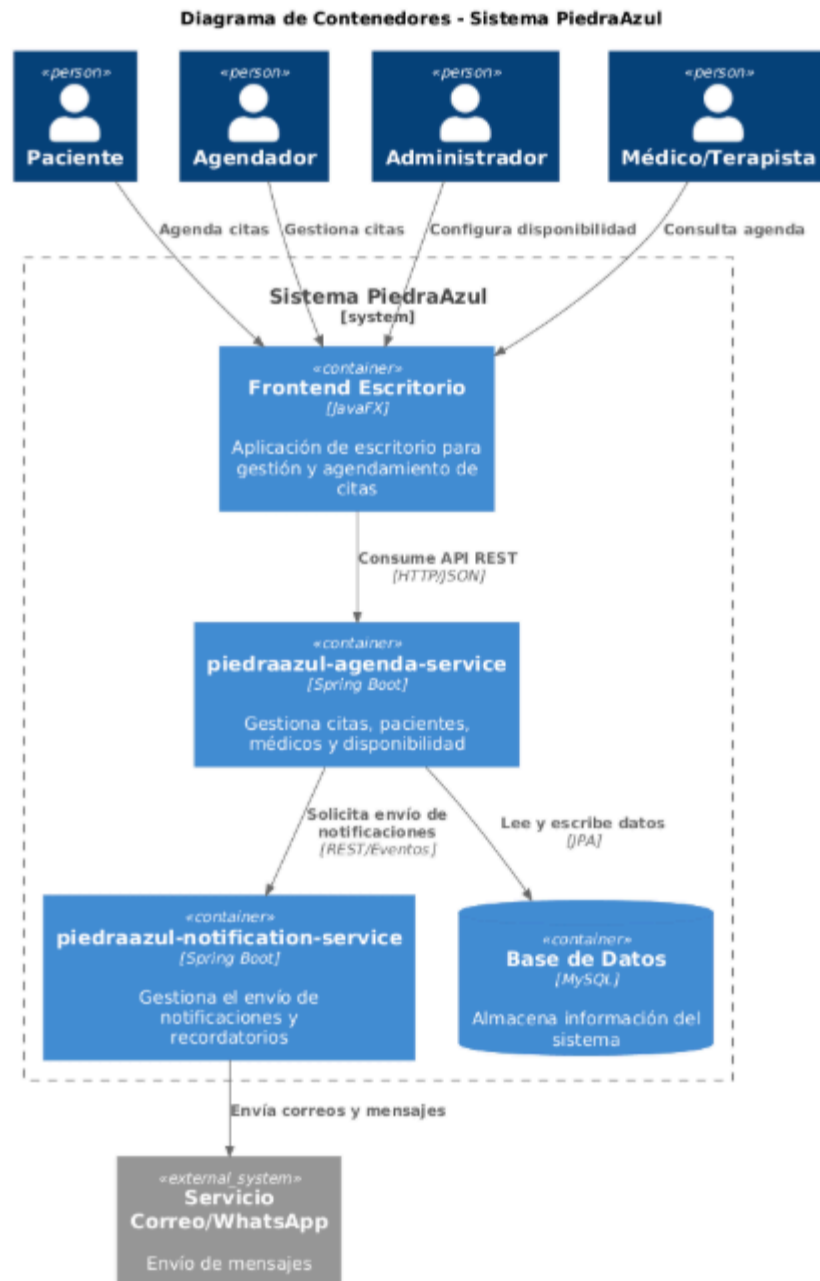
Descripción del diagrama de contexto:

El diagrama de contexto representa la interacción general entre los actores principales y el sistema PiedraAzul. Los actores identificados son el paciente, el agendador, el administrador y el médico o terapeuta. Cada uno interactúa con el sistema según su rol: el paciente agenda citas médicas, el agendador gestiona citas, el administrador configura horarios y disponibilidad, y el médico o terapeuta consulta la agenda médica.

consulta su agenda.

Además, el sistema se relaciona con servicios externos de notificación, como correo electrónico o WhatsApp, los cuales permiten enviar confirmaciones y recordatorios asociados a las citas médicas. Este nivel permite observar el sistema como una unidad completa y entender su relación con usuarios y servicios externos.

Diagrama de Contenedores – Modelo C4 (Nivel 2)



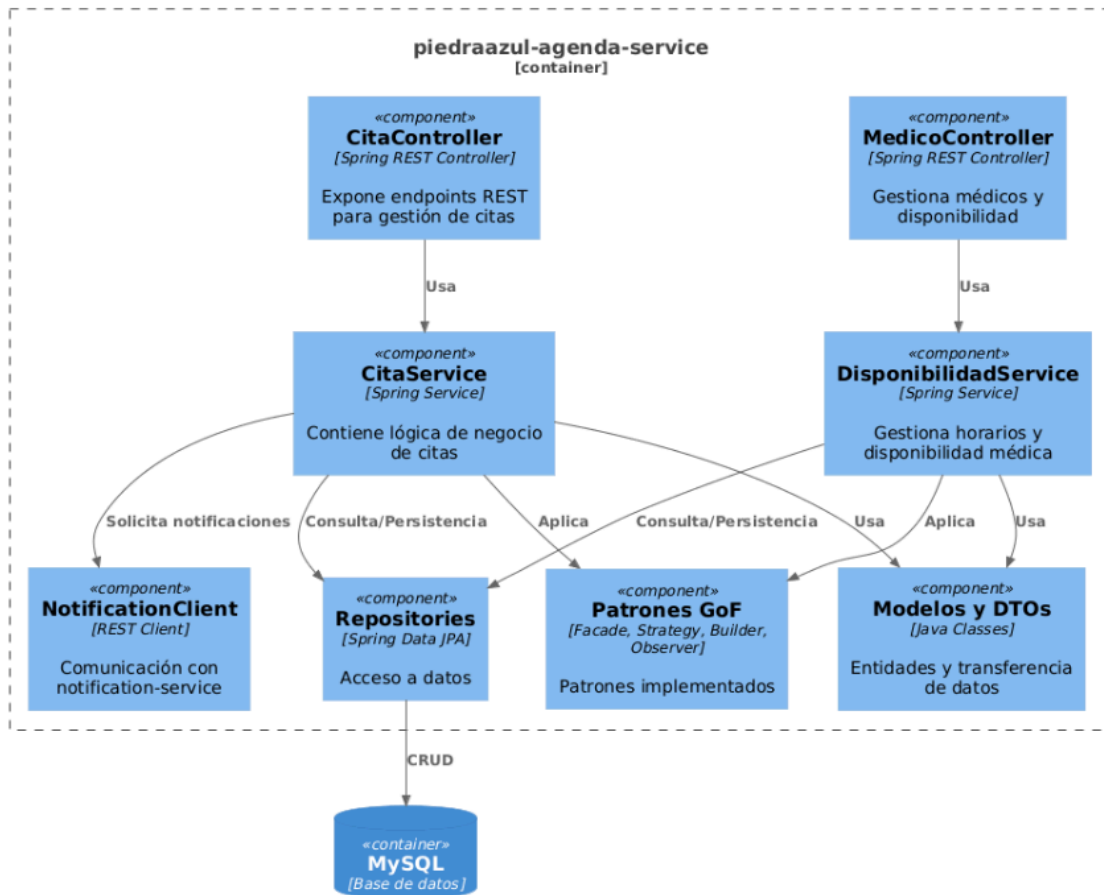
Descripción del diagrama de contenedores:

El diagrama de contenedores muestra la estructura principal del sistema PiedraAzul. La solución está compuesta por un frontend de escritorio, una API REST, el microservicio de agenda, el microservicio de notificaciones y una base de datos MySQL.

El frontend de escritorio permite la interacción de los usuarios con el sistema. La API REST actúa como punto de comunicación entre la interfaz gráfica y los servicios internos. El microservicio de agenda se encarga de gestionar citas, disponibilidad y reglas de negocio relacionadas con el agendamiento. El microservicio de notificaciones procesa los eventos generados por el sistema y envía mensajes de confirmación o recordatorio. La base de datos almacena la información necesaria para la operación del sistema, como usuarios, médicos, horarios, disponibilidad y citas.

Diagrama de Componentes – Agenda Service (Nivel 3)

Diagrama de Componentes - piedraazul-agenda-service



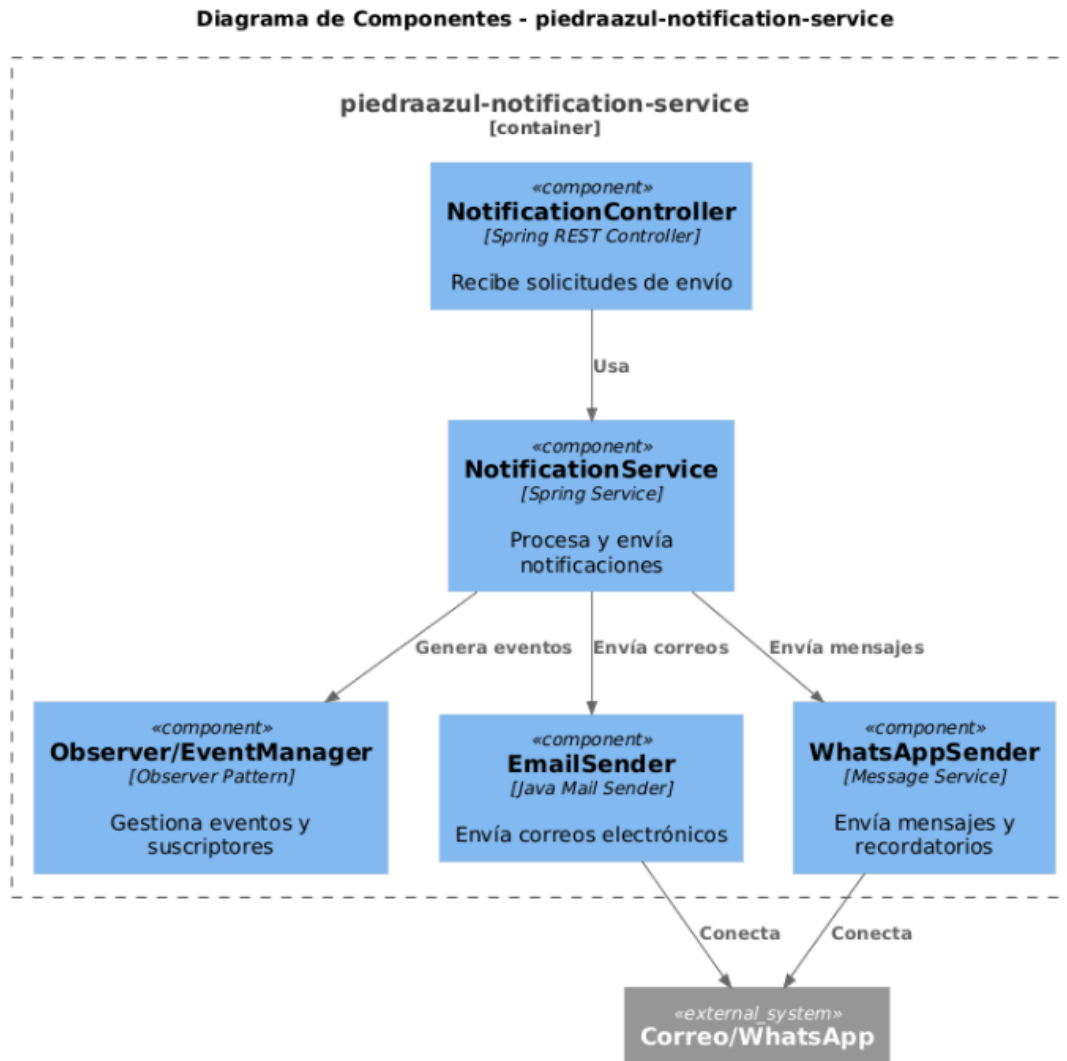
Descripción del diagrama de componentes del microservicio de agenda:

El diagrama de componentes del microservicio de agenda muestra la organización interna del servicio encargado de gestionar el proceso de agendamiento de citas. Este microservicio contiene controladores REST que reciben las solicitudes del frontend, servicios de negocio que validan disponibilidad y gestionan citas, repositorios encargados del acceso a datos y componentes asociados a los patrones de diseño implementados.

Dentro de este microservicio se aplican reglas de negocio como la validación de horarios disponibles, la prevención de cruces entre citas y la creación de eventos cuando una cita es registrada. Esta estructura permite separar responsabilidades,

facilitar las pruebas unitarias y mantener el código organizado.

Diagrama de Componentes – Notification Service (Nivel 3)



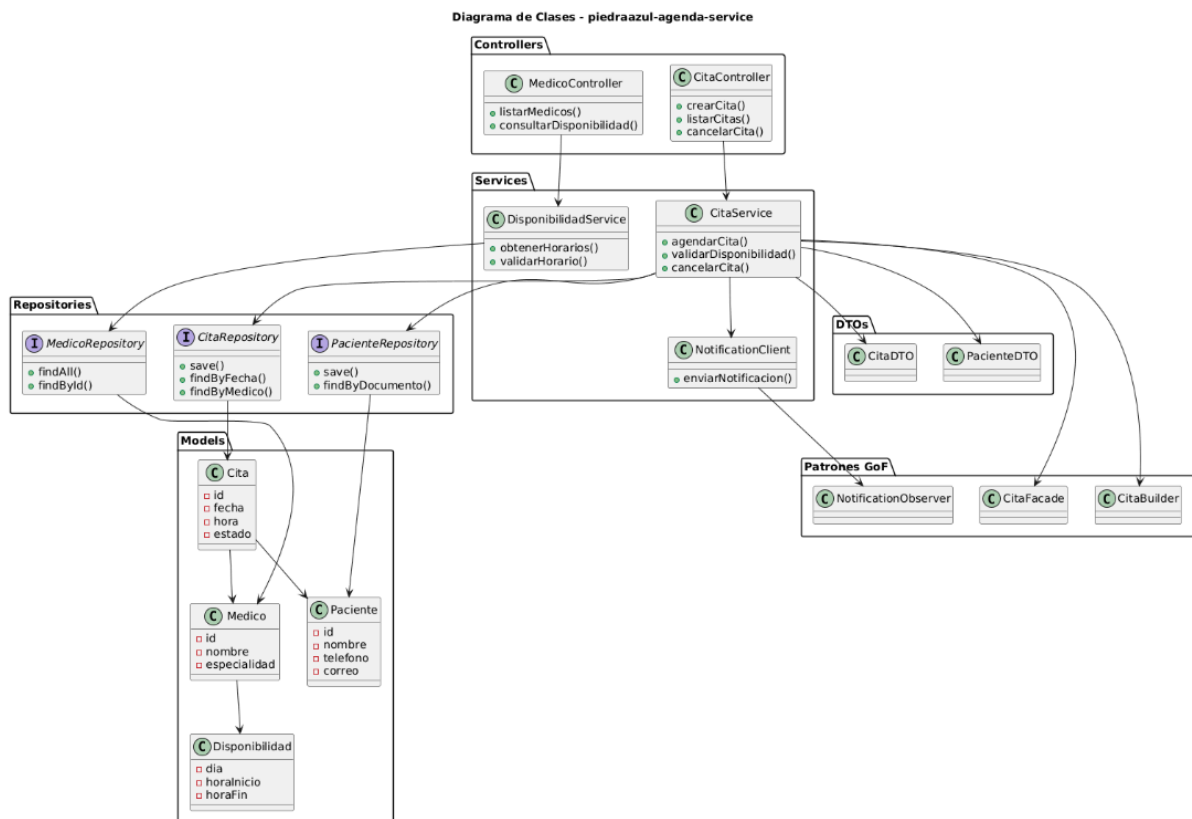
Descripción del diagrama de componentes del microservicio de notificaciones:

El diagrama de componentes del microservicio de notificaciones representa la estructura interna del servicio encargado de procesar y enviar mensajes relacionados con las citas médicas. Este microservicio recibe eventos generados

por el sistema, como la creación de una cita, y posteriormente ejecuta la lógica necesaria para enviar una notificación al paciente.

El servicio contiene componentes como el controlador de notificaciones, el servicio de procesamiento, el gestor de eventos y los adaptadores para canales externos como correo electrónico o WhatsApp. Esta separación permite desacoplar el envío de mensajes de la lógica principal de agendamiento, mejorando la mantenibilidad y permitiendo agregar nuevos canales de notificación en el futuro.

Diagrama de Código / Clases (Opcional – Nivel 4)

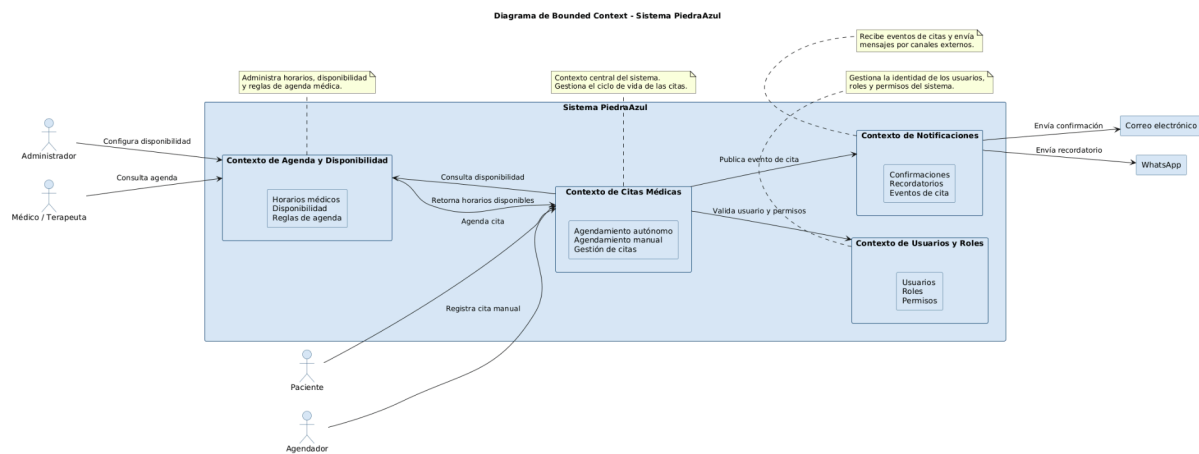


Descripción del diagrama de código o clases:

El diagrama de código muestra una vista más detallada de algunas clases utilizadas en la implementación del sistema. En este nivel se pueden observar las relaciones entre controladores, servicios, repositorios, modelos y clases asociadas a los patrones de diseño GoF.

Aunque este nivel es opcional dentro del modelo C4, se incluye para evidenciar cómo las decisiones arquitectónicas fueron llevadas al código. Este diagrama permite identificar la separación de responsabilidades, la aplicación de principios SOLID y la integración de patrones como Builder, Factory, Facade, Adapter, Strategy y Observer.

Bounded Context



Listado de patrones de diseño implementados

Durante la segunda iteración del sistema PiedraAzul se implementaron diferentes patrones de diseño GoF con el objetivo de mejorar la organización del código, reducir el acoplamiento entre componentes, facilitar la reutilización de lógica y permitir una mejor mantenibilidad del sistema.

1. Builder — Patrón creacional

El patrón Builder fue utilizado para facilitar la construcción de objetos complejos relacionados con las citas médicas y las respuestas del sistema. En lugar de utilizar constructores con muchos parámetros, este patrón permite construir los objetos paso a paso, haciendo que el código sea más legible y fácil de mantener.

En el contexto del sistema PiedraAzul, este patrón se aplicó en la creación de objetos relacionados con el agendamiento, donde se requiere asociar información como paciente, médico o terapeuta, fecha, hora, estado de la cita y datos adicionales de confirmación.

2. Factory Method — Patrón creacional

El patrón Factory Method fue implementado para centralizar la creación de objetos o servicios según el tipo de operación requerida. Esto permite que las clases principales no dependan directamente de implementaciones concretas, sino de una fábrica encargada de decidir qué objeto crear.

En el sistema, este patrón se relaciona con la creación de componentes asociados al agendamiento o a las notificaciones, permitiendo seleccionar una implementación específica sin modificar la lógica principal del sistema.

3. Facade — Patrón estructural

El patrón Facade fue utilizado para simplificar la comunicación entre los controladores y los servicios internos del sistema. En lugar de que un controlador dependa directamente de múltiples clases o servicios, se utiliza una fachada que centraliza las operaciones necesarias.

En el contexto del sistema PiedraAzul, este patrón facilita operaciones como registrar una cita, validar disponibilidad, consultar información del paciente y generar eventos de notificación. De esta forma, se reduce el acoplamiento y se mejora la claridad del flujo de trabajo.

4. Adapter — Patrón estructural

El patrón Adapter fue implementado para permitir la comunicación entre componentes internos del sistema y servicios externos o interfaces diferentes. Este patrón permite adaptar una interfaz existente a otra esperada por el sistema, sin

modificar la lógica principal.

En el sistema PiedraAzul, este patrón se aplica en la integración con canales externos de notificación como correo electrónico o WhatsApp. Gracias al adaptador, el microservicio de notificaciones puede enviar mensajes sin depender directamente de la implementación específica de cada proveedor externo.

5. Strategy — Patrón de comportamiento

El patrón Strategy fue utilizado para definir diferentes formas de validar o gestionar reglas de disponibilidad y agendamiento. Este patrón permite encapsular varias estrategias y seleccionar la más adecuada según el contexto.

En el sistema, puede aplicarse para manejar diferentes reglas de disponibilidad médica, validaciones de horarios, tipos de cita o criterios de asignación de profesionales. Esto permite cambiar o ampliar las reglas de negocio sin modificar directamente las clases principales.

6. Observer — Patrón de comportamiento

El patrón Observer fue implementado para gestionar eventos dentro del sistema, especialmente en el proceso de notificaciones. Cuando ocurre una acción importante, como la creación de una cita médica, el sistema genera un evento que puede ser recibido por otros componentes interesados.

En el contexto de PiedraAzul, este patrón permite que el microservicio de agenda notifique la creación de una cita sin depender directamente del microservicio de notificaciones. Esto mejora el desacoplamiento entre servicios y facilita la incorporación de nuevos observadores en el futuro.

URL del video de YouTube

URL del repositorio GIT

<https://github.com/bCharry15/Sprint2-PiedrAzul.git>